

Server-Factory-Core-Framework

모든 **Server Factory**의 공통 엔진

개요

Core Framework는 Server Factory 제품군의 프로비저닝 도구를 지탱하는 Kotlin 기반 프레임워크입니다. 이 프레임워크는 Mail Server Factory와 같은 프로젝트들이 기반으로 삼는 공통 엔진과 추상화를 제공하여, 각 '팩토리'가 프로비저닝 기본 요소를 재구현하는 대신 검증된 하나의 토대를 재사용할 수 있도록 합니다.

간략 설명

Server Factory 생태계의 기반이 되는 공유 Kotlin 프레임워크입니다. 하위 팩토리(Mail Server Factory, Web Service Factory, SonarQube Factory 등)에서 사용하는 공통 프로비저닝 엔진, 연결 추상화, 설치 단계 메커니즘을 제공합니다.

상세 설명

Core Framework는 전체 Server-Factory 제품군을 가능하게 하는 숨은 공학의 결정체입니다. 개별 '팩토리' 제품(Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory 등)은 모두 이 재사용 가능한 엔진 위에 구축됩니다. Server Factory의 접근 방식은 선언형입니다. 사용자는 원하는 인프라를 구성 파일로 기술하면, 팩토리가 이를 해석해 대상 시스템에 소프트웨어를 설치하고 초기화합니다. 그리고 이러한 패턴의 공통 메커니즘이 바로 Core Framework에 구현되어 있습니다. 여기에는 모든 종류의 대상 시스템에 연결하기 위한 연결 및 전송 추상화, 소프트웨어가 어떻게 프로비저닝되는지를 정의하는 설치 단계 모델, 그리고 각 팩토리가 개별적으로 구현해야 했을 공통 인프라가 포함됩니다.

Core Framework는 다중 제품 툴체인이 결국 마주하게 되는 구조적 질문에 대한 해답입니다. “공통 엔진은 어디에 두어야 하는가?”라는 질문에 대한 올바른 답을 한 번만 찾아내면, 제품군이 네 가지 미묘하게 다른 프로비저너로 분열되는 대신 일관성을 유지할 수 있습니다. 이 프레임워크로 공통

기능을 집중함으로써, 제품군 전체에서 프로비저닝 로직의 중복을 방지하고 동작의 일관성을 확보합니다. Core Framework에서 개선된 연결 유형이나 설치 기본 요소는 모든 하위 팩토리에 즉시 적용됩니다.

코드베이스의 거의 대부분은 Kotlin(약 990KB 규모의 Kotlin 코드에 얇은 Shell 레이어)로 구성되어 있으며, 이는 스크립트 모음이 아닌 코드 라이브러리라는 역할을 반영합니다. 하위 저장소들은 이 프레임워크를 핵심 의존성으로 참조합니다(Parallels-Utils, Qemu-Utils, Utils, Definitions 팩 등은 모두 Core Framework 저장소를 생태계의 허브로 연결합니다). README는 의도적으로 간결하게 작성되어 있으며, 다른 프로젝트를 위한 인프라로서 version.txt/version_code.txt를 통해 버전 관리가 이루어집니다. 또한 Core Framework는 이후의 AI 작업보다 앞서 개발되어 조직의 성숙한 DevOps 틀체인 유산의 일부로 자리 잡았습니다.

개발 이유

모든 프로비저닝 도구는 동일한 핵심 기능을 필요로 합니다. 대상 시스템에 연결하는 방법과 소프트웨어를 설치/구성하는 단계가 바로 그것입니다. 이를 제품별로 재구현하면 동작이 파편화되고 버그가 증가합니다. Core Framework는 이러한 기능을 중앙 집중화하여 모든 팩토리가 하나의 신뢰할 수 있는 엔진을 공유하도록 합니다.

혁신적인 이유

Core Framework는 제품군 전체에서 가장 높은 레버리지 포인트입니다. 여기서 강화된 연결 유형이나 개선된 설치 기본 요소는 즉시 모든 팩토리에 적용되어, 전체 틀체인이 하나의 투자에서 복리 효과를 얻게 됩니다. 이는 “한 번 구축하고, 어디서나 재사용한다”는 철학을 인프라 자동화의 기반 레이어에 적용한 결과입니다. 바로 이 지점에서 수정한 내용이 하위 모든 시스템에 영향을 미치기 때문에, 올바른 곳에 수정을 가하면 모든 것이 개선됩니다.

콘텐츠

혁신적인 점

- 연결 및 설치 단계 로직을 추상화한 단일 재사용 가능한 프로비저닝 프레임워크.
- 엔진(코어 프레임워크)과 제품별 팩토리 간의 명확한 분리.
- 재현 가능한 배포를 위한 버전 고정 배포(version.txt/version_code.txt).

도전 과제 및 해결 방안

- 프로비저닝 로직의 중복 방지: 모든 팩토리에서 공유하는 공통 프레임워크로 추출하여 해결.

- 제품 간 일관된 동작: 연결 유형과 단계가 어디서든 동일하게 작동하도록 공통 추상화를 적용하여 해결.
- (미검증): 특정 내부 API는 공개 README에 문서화되지 않았으며, “팩토리에서 공유하는 프레임워크”라는 점 외에는 인터페이스 세부 사항이 미검증 상태로 간주됨.

기술 스택(이유 및 사용 방식)

- **Kotlin** — 전체 프레임워크(약 990KB); Server Factory 계열의 언어.
- **Shell** — 최소한의 지원 스크립트.
- **Gradle** — 빌드 도구 체인(계열의 ./gradlew 사용과 일관성 유지).

참고: GitHub는 Server-Factory 조직 내 포크 저장소로 표시됨. AI 중심이 아닌 프로비저닝 도구 체인의 기반으로 제시됨.